

(the diagonal line) at the time any drawing is rendered, but only after the drawing operation is complete.

After the translation has been applied, you can perform the normal drawing operations to create the diagonal line and render it to the canvas by calling the `context.stroke` method. Finally, you restore the context to its clean original state, so that future canvas operations are performed without the translation that was applied in this operation.

Canvas Coordinates

The coordinates in a canvas start at $x=0, y=0$ in the upper-left corner—which is referred to as the origin—and increase (in pixels) horizontally over the x -axis and vertically over the y -axis. Along the X -axis, values increase towards the right edge of the canvas. Along the Y -axis, values increase towards the bottom edge of the canvas.

The coordinate diagram can be drawn with a `<canvas>` element. It will comprise

- ▶ a set of off-white vertical lines
- ▶ a set of off-white horizontal lines
- ▶ two black horizontal lines
- ▶ two small black diagonal lines that form an arrow
- ▶ two black vertical lines
- ▶ two small black diagonal lines that form another arrow
- ▶ the letter “x”
- ▶ the letter “y”
- ▶ the text “(0, 0)” near the upper-left corner
- ▶ the text “(500, 375)” near the lower-right corner
- ▶ a dot in the upper-left corner, and another in the lower-right corner

First, we need to define the `<canvas>` element itself. The `<canvas>` element defines the width and height, and the id so we can find it later. Then we need a script to find the `<canvas>` element in the DOM and get its drawing context.

- `<canvas id="CoodSys" width="500" height="375"></canvas>`
- `var CoodSys _ canvas = document.getElementById("CoodSys");`
- `var context = CoodSys _ canvas.getContext("2d");`

Drawing Paths

Paths in the HTML5 Canvas API represent any shape you care to render. Our original line example was a path and we can complicate it as we desire,